TECHNICAL ARCHITECTURE & PROJECT EXPLANATION GUIDE

Shadow AI Prompt Guardian (SENTINEL)

Browser-Level Real-Time Detection and Risk Assessment of Unauthorized AI Usage

Authors: Sreegokul S., Roopesh S. N., Ram Sakthi Sharumathi M., Sanjeev Kumar P., Saravana Kumar M.
Affiliation: Department of CSE (IoT & Cyber Security), SNS College of Engineering, Tamil Nadu, India

1. The Problem Space & Architectural Paradigm Shift

Modern enterprise employees increasingly leverage public Generative AI interfaces—such as ChatGPT, Claude, Gemini, Perplexity, and Cohere—to accelerate drafting, coding, and debugging. While productivity rises, it introduces a severe corporate threat termed **"Shadow AI":** the unmanaged, unauthorized transmission of proprietary corporate assets through public AI tools without governance or oversight.

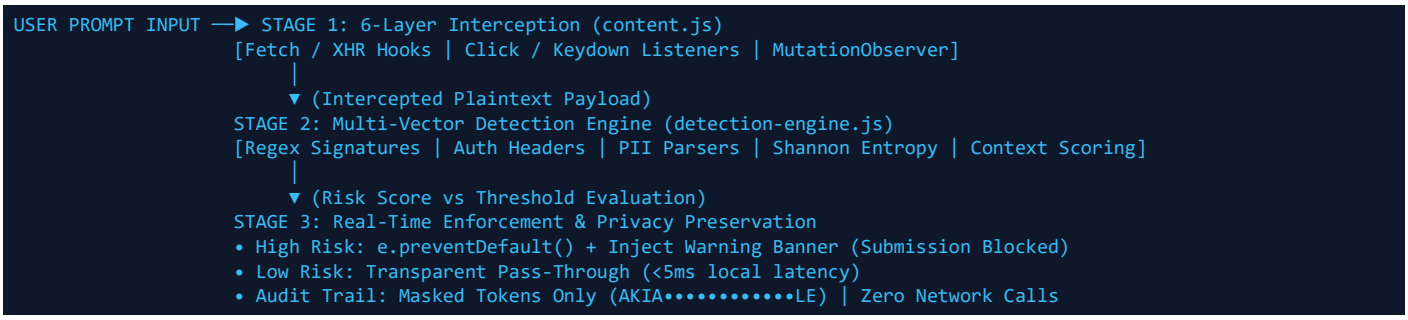
> **CRITICAL VULNERABILITY: THE IRREVERSIBILITY OF SHADOW AI LEAKS**
>
> Unlike traditional data transmission or file sharing, prompt submissions to public generative-AI interfaces are permanent and irreversible. Once proprietary source code, credentials, or customer PII are submitted, the data cannot be revoked or erased from remote inference caches. The information becomes subject to the host platform's data retention policies and model retraining pipelines. Effective mitigation must prevent sensitive content from exiting the browser boundary, rather than detecting its loss after the fact.

Why Legacy Defensive Solutions Fail:

- **Network & DNS-Level Blocklists:** Traditional firewalls and DNS sinkholes filter by destination domain. They check where traffic is going, completely ignoring what is being transmitted. Consequently, newly launched, unlisted, or mirror AI endpoints remain fully accessible, while blanket domain bans break legitimate employee workflows.
- **SaaS & CASB Discovery Platforms:** Cloud Access Security Brokers (CASB) inspect OAuth grants and email accounts after usage occurs. Because they operate post-facto, they notify security teams days or weeks after data has already departed corporate custody.
- **The Guardian's Solution:** Client-side, pre-transmission content inspection. By deploying a Manifest V3 browser extension, the inspection engine operates directly within the browser execution context, analyzing prompt text synchronously at keystroke and submission time with zero external network dependencies.

2. Three-Stage Technical Pipeline Architecture

The Shadow AI Prompt Guardian executes via a modular, synchronous 3-stage pipeline implemented in client-side JavaScript, ensuring sub-5ms evaluation without introducing human-perceptible input latency:



DETAILED ENGINE IMPLEMENTATION & EMPIRICAL BENCHMARKS

Stage 1: The 6-Layer Synchronous Interception Engine (content.js)

Modern AI web frontends are single-page applications (SPAs) built with React, Next.js, and custom event handlers that bypass standard HTML form submission. To guarantee comprehensive capture across ChatGPT, Claude, Gemini, Perplexity, and Cohere, the extension deploys six concurrent interception hooks:

1.1 Fetch API Hook: Monkey-patches window.fetch to inspect outgoing request bodies immediately prior to serialization.

1.2 XMLHttpRequest (XHR) Hook: Overrides XMLHttpRequest.prototype.send to capture asynchronous AJAX transmissions.

1.3 Click Event Detection: Attaches capture-phase event listeners (useCapture: true) to AI submit buttons, preempting web handlers.

1.4 Keyboard Enter Detection: Intercepts Enter and Shift+Enter keydown events within textareas and prompt input fields.

1.5 Form Submission Hook: Hooks standard HTML form submit events for legacy server-rendered interfaces.

1.6 MutationObserver: Continuously tracks dynamic DOM additions, observing contenteditable divs and newly rendered input elements in reactive SPAs.

Stage 2: Multi-Vector Detection Engine (detection-engine.js)

The extracted text payload is evaluated across five complementary inspection layers entirely in local memory:

2.1 Deterministic Regex Signatures: Tailored high-precision regular expressions targeting 10+ cloud platforms (AWS AKIA..., GitHub ghp_..., OpenAI sk-..., Stripe sk_live_..., Slack xoxb-..., Azure, Google OAuth, Twilio, GitLab).

2.2 Credential & Auth Parsing: Detects Bearer authorization tokens, Basic Auth base64 strings, and explicit password assignment syntax (e.g., password=, passwd:, api_secret=).

2.3 Syntactic PII Detection: Validates structural integrity of emails, US and international phone numbers, Social Security Numbers (SSNs), and IPv4/IPv6 addresses.

2.4 Shannon Entropy Mathematical Metric: Calculates character probability $H = -\sum P(x_i) \log_2(P(x_i))$ across token substrings. High-randomness strings (≥20 characters, $H \geq 4.0$ bits/char) are flagged as unknown secrets, cryptographic hashes, or unclassified API tokens.

2.5 Context-Aware Score Suppression: Suppresses false positives by dampening risk scores when sensitive keywords occur within educational queries (e.g., 'What is an API key?') or inert code syntax.

Stage 3: Real-Time Enforcement & Zero-Exfiltration Design

If the composite risk score equals or exceeds the blocking threshold, the extension invokes e.preventDefault() and e.stopImmediatePropagation(), immediately halting submission. A warning banner is injected directly above the prompt box, detailing the detected risk categories. Crucially, the extension performs zero external network telemetry, stores zero raw prompt text, and records only irreversibly masked audit tokens (e.g., AKIA••••••••••••LE), guaranteeing total enterprise confidentiality.

3. Empirical Evaluation & Stress-Test Benchmarks

The system was evaluated through two distinct testing methodologies: a primary categorized enterprise benchmark and a large-scale combinatorial adversarial stress test designed to probe detection boundaries.

Evaluation Scope	Prompts	Categories Evaluated	Accuracy	Recall / F1	Key Empirical Findings
Primary Categorized Dataset	140	Clean (45), API Keys (30), PII (23), Terms (23), Pwd (19)	**100.0%**	**1.00 / 1.00**	Zero FP & Zero FN; perfect classification across recognizable syntaxes.
Combinatorial Stress Test	1,000	15 templates × 15 contexts × 8 credential styles	**95.4%**	**95.4% / —**	Intercepted 954/1000 TP; surfaced 46 FN across two reproducible boundaries.

Characterization of the Two Failure Modes:

- **Label-Style Secret Phrasing (Syntactic Boundary):** When credentials are introduced via plain labels without assignment operators (e.g., 'Secret: <key>' instead of 'password=<key>'), standard assignment regexes are bypassed. This accounted for the majority of the 46 missed cases.
- **Context-Aware Score Suppression (Damping Side-Effect):** While context scoring effectively eliminates false positives on benign queries, query-style technical prefixes (e.g., 'Help me debug this token...') occasionally dampen risk scores of genuine secrets below the critical blocking threshold.

DEFENSE GUIDE, FREQUENTLY ASKED QUESTIONS & ROADMAP

4. Frequently Asked Technical Questions (Defense & Review Guide)

Q1: Why not use a cloud LLM to detect prompt leaks?

A1: Three critical reasons: (1) Latency: Cloud LLMs introduce 500ms–2000ms latency, ruining interactive user typing. (2) Cost: Per-token API costs scale unsustainably with enterprise prompt volume. (3) Privacy Paradox: Sending prompts to an external cloud model to check for leaks violates the very confidentiality standard being enforced. Our client-side JS engine executes in <5ms locally with zero network egress.

Q2: Why can't a corporate proxy or Web Application Firewall (WAF) solve this?

A2: Modern browser traffic utilizes TLS 1.3 with certificate pinning or WebSocket connections. Network proxies cannot inspect encrypted payloads without deploying invasive enterprise root certificates. Furthermore, proxies lack DOM context to distinguish between active typing, reading, or drafts, whereas client-side hooks capture data at the moment of submission.

Q3: Why choose Manifest V3 over Manifest V2?

A3: Manifest V3 is Google Chrome's modern extension standard. It enforces enhanced security by prohibiting arbitrary remote code execution, migrates background pages to ephemeral service workers, and provides isolated world script execution that prevents malicious website scripts from tampering with the Guardian's detection logic.

Q4: How does Shannon Entropy prevent false positives on normal text?

A4: Natural language English text exhibits an entropy of ~1.5 to 3.0 bits/character due to character frequency distributions. Cryptographic keys and base64 tokens exhibit high uniformity and randomness (H >= 4.0 bits/char). By pairing this mathematical threshold with a minimum length constraint (>= 20 chars), common words are excluded while random keys are flagged.

5. Identified Limitations & Engineering Roadmap

→ **Multi-Turn Context Window Analysis:** Aggregating conversational history across multi-message threads to prevent 'slow-drip' data exfiltration split across sequential prompts.

→ **Client-Side Document Scanning:** Extending synchronous inspection to local file uploads (PDF, CSV, DOCX) before transfer to AI platforms.

→ **Embedded Lightweight ML (TensorFlow.js):** Integrating client-side on-device models to detect paraphrased or semantic leaks without cloud calls.

→ **Tiered Severity Classification:** Implementing graduated alert levels (Critical Block, Warning Confirmation, Informational Notice) to optimize user workflow and eliminate alert fatigue.

6. Platform Support & Architectural Implementation Matrix

AI Platform	Input Container Type	Interception Mechanism	Operational Status
OpenAI ChatGPT	contenteditable / textarea #prompt-textarea	Fetch hook + Send button click + Keydown	**Verified Active**
Anthropic Claude	div.ProseMirror contenteditable	MutationObserver + Fetch hook + Enter key	**Verified Active**
Google Gemini	rich-textarea, div[role='textbox']	XHR/Fetch hook + Click event preemption	**Verified Active**
Perplexity AI	textarea[placeholder*='Ask anything']	Fetch API monkeypatch + Form submit	**Verified Active**
Cohere Coral	div.coral-input / input[type='text']	DOM MutationObserver + Keydown hook	**Verified Active**